# DevOps Engineering Capstone Report

Comprehensive Final Deliverables: Capstone 1 ("Ship It") & Capstone 2 ("Keep It Running")

**Student:** Paul (paulycarp) | **Instructor:** Toluwalase Adedotun | **Program:** DevOps Professional Capstone | **Date:** October 2026

## 1. Executive Summary & Verification State

This report provides the formal submission for both DevOps Capstone projects. All components have been implemented, tested, verified end-to-end, and published to GitHub with complete documentation, Infrastructure as Code, CI/CD pipelines, and Site Reliability Engineering (SRE) operational artifacts.

> **Live Verification Highlights:** All microservices were deployed and tested on a 2-node Kubernetes cluster (Kind) provisioned via Terraform. Capstone 1 container images were built, hardened (non-root, pinned tags), scanned with Trivy, and packaged into Helm charts with verified rollback capabilities. Capstone 2 observability stack (Prometheus, Grafana Golden Signals, Fluent Bit, Loki, OpenTelemetry, and Jaeger) was deployed, and a live incident drill was executed and remediated via the documented SRE runbook.

📦 **Capstone 1 Repository: Example Voting App**

**URL:** https://github.com/paulycarp/example-voting-app | **Branch:** `main`
**Focus:** Container security hardening, Infrastructure as Code (Terraform for Local Kind & AWS EKS), GitHub Actions CI/CD with Trivy CVE scanning, Helm chart packaging and live rollback.

🛡️ **Capstone 2 Repository: Stan's Robot Shop**

**URL:** https://github.com/paulycarp/robot-shop | **Branch:** `master`
**Focus:** Kubernetes microservices deployment, APM decoupling, full observability stack (Prometheus, Grafana, Loki, Jaeger), SLO & Error Budget policy, Operational Runbook, and Blameless Postmortem.

## 2. Capstone 1: "Ship It" Implementation Details

### A. Hardened Containerization

The original codebase contained insecure, unpinned base images running as the root user. All Dockerfiles were refactored according to production security standards:

| Microservice | Base Image | Security Hardening Applied | Execution User |
| --- | --- | --- | --- |
| **vote** (Python Flask) | `python:3.11.11-slim` | Multi-stage build, no package cache, pinned version | appuser (UID 10001) |
| **result** (Node.js) | `node:18.20.7-slim` | Production dependencies only ( `npm ci --omit=dev` ), tini init | node (UID 1000) |
| **worker** (.NET 7) | `mcr.microsoft.com/dotnet/runtime:7.0` | Multi-stage SDK compilation, optimized self-contained binary | appuser (UID 10001) |

### B. Infrastructure as Code (IaC)

- **Local Cluster ( `terraform/local/` ):** Provisions a 2-node Kind cluster ( `voting-app-cluster-control-plane` and `voting-app-cluster-worker` ) mapping NodePorts directly to host ports ( `8080` , `8081` , `3000` , `9090` ). Verified repeatable with clean destroy and apply.
- **Cloud Cluster ( `terraform/aws/` ):** Production-ready AWS EKS configuration deployed inside the AWS account's **Default VPC** with IAM OIDC provider, managed node groups, and zero secret leakage.

### C. Automated CI/CD Pipeline (GitHub Actions)

The workflow ( `.github/workflows/ci-cd.yaml` ) triggers on pushes to `main` and Pull Requests:

1. **Automated Testing:** Python Flake8 linting, Node.js dependency validation, and .NET compilation checks.
2. **Vulnerability Scanning:** Aqua Security **Trivy** scans container images for `CRITICAL` and `HIGH` CVEs before publishing.
3. **Container Publishing:** Builds and pushes immutable images tagged with the Git commit SHA ( `${{ github.sha }}` ) and `latest` to GitHub Container Registry ( `ghcr.io` ).
4. **Helm Linting:** Validates chart syntax and template rendering across `helm/voting-app/` .

### D. Helm Packaging & Live Rollback

- Single unified chart in `helm/voting-app` parameterizing replicas, images, resources, and NodePorts.
- **Rollback Verification:** Successfully injected a failing image tag ( `--set vote.image.tag=does-not-exist` ), observed deployment degradation ( `ImagePullBackOff` ), and executed `helm rollback voting-app 1` to restore zero-downtime service.

## 3. Capstone 2: "Keep It Running" Implementation Details

### A. Kubernetes Architecture & APM Decoupling

Stan's Robot Shop (14 microservices) was deployed to the `robot-shop` namespace. All proprietary vendor APM agents (Instana) were completely decoupled from deployments, replacing closed-source monitoring with open-source Cloud Native Computing Foundation (CNCF) standards.

### B. Full Observability Stack Architecture

| Observability Pillar | Technology | Endpoint / Ingress | Purpose & Capabilities |
|---|---|---|---|
| **Metrics** | Prometheus (Operator) | `http://localhost:9090` | Cluster metrics, kube-state-metrics, node-exporter, and SLO recording rules. |
| **Dashboards** | Grafana 13 | `http://localhost:3000` | **Golden Signals Dashboard as Code** tracking Latency, Traffic, Errors, and Saturation. |
| **Logs** | Fluent Bit + Loki | `http://loki:3100` | DaemonSet log collector tailing container logs, parsing Kubernetes metadata, and indexing in Loki. |
| **Traces** | OpenTelemetry + Jaeger | `http://localhost:16686` | OTel Collector accepting OTLP gRPC/HTTP spans and visualizing end-to-end distributed traces in Jaeger. |

### C. SRE Artifacts & Deliverables

- **SLO & Error Budget Policy ( `docs/slo-error-budget-policy.md` ):** Establishes explicit availability targets (99.0% for e-commerce services, 99.5% for Checkout/Payment) over a 30-day rolling window. Defines strict multi-window multi-burn-rate thresholds and feature freeze triggers upon 100% budget depletion.
- **PrometheusRule Alerting ( `monitoring/slo/prometheus-slo-rules.yaml` ):** Automates alerting for service degradation ( `RobotShopServiceDegraded` ) routing to Alertmanager and linking directly to the operational runbook.
- **Operational Runbook ( `docs/runbook.md` ):** Provides step-by-step triage, log inspection via Loki, trace analysis via Jaeger, rollback commands, and mitigation steps for on-call engineers.
- **Blameless Postmortem ( `docs/blameless-postmortem.md` ):** Comprehensive incident report detailing incident timelines, root cause analysis using the 5 Whys methodology, and preventative action items with priority assignments.

### D. Practical Incident Drill & Live Recovery

> **Simulated Incident Drill:** Failure was deliberately injected into the `payment` service ( `kubectl scale deployment payment --replicas=0` ). Traffic drop and error rate surges were observed in real time on

> Grafana's *Stan's Robot Shop - Golden Signals* dashboard. Prometheus availability alert transitioned to firing state. The on-call triage process from `docs/runbook.md` was executed, scaling capacity back to 1 replica. Prometheus and Grafana verified full recovery to 100% green availability.

## 4. Syllabus Requirements Verification Checklist ("Done Means")

| Capstone | Deliverable Item | Status | Repository Location |
|---|---|---|---|
| Capstone 1 | Hardened Dockerfiles (non-root, pinned base tags) | COMPLETED | `vote/`, `result/`, `worker/` |
| Capstone 1 | Terraform IaC (Kind Local + AWS EKS Default VPC) | COMPLETED | `terraform/local/`, `terraform/aws/` |
| Capstone 1 | GitHub Actions CI/CD with Trivy CVE Scanning | COMPLETED | `.github/workflows/ci-cd.yaml` |
| Capstone 1 | Helm Chart Packaging with Live Rollback Verification | COMPLETED | `helm/voting-app/` |
| Capstone 2 | Robot Shop Kubernetes Deployment & APM Decoupling | COMPLETED | `K8s/helm/` |
| Capstone 2 | Golden Signals Dashboard Exported as Code | COMPLETED | `monitoring/dashboards/robot-shop-golden-signals.json` |
| Capstone 2 | Logging & Tracing Infrastructure | COMPLETED | `monitoring/logging/`, `monitoring/tracing/` |
| Capstone 2 | SLO & Error Budget Policy Document | COMPLETED | `docs/slo-error-budget-policy.md` |
| Capstone 2 | Operational Runbook Linked from Alerts | COMPLETED | `docs/runbook.md` |
| Capstone 2 | Blameless Postmortem Report | COMPLETED | `docs/blameless-postmortem.md` |
| Capstone 2 | Live Incident Simulation & Recovery Drill | VERIFIED | Observed in Prometheus & Grafana |